

OptOP User Guide

Optimal Order Parameters — Fortran/MPI Order-Parameter Computation & XGBoost Phase Classification

OptOP computes structural order parameters from LAMMPS trajectory files and trains XGBoost models to classify phases from them. The order-parameter engine is Fortran, parallelised with MPI over trajectory frames and optionally over atoms within a frame. It was built for water/ice and gas-hydrate discrimination but works for any `.lammprj` trajectory.

*This guide replaces the earlier **OP_ML User Guide**. The package was renamed and restructured: the import path is now **optop**, the Fortran extension compiles automatically at install time, and two MPI axes are available. Section 15 lists everything that changed.*

1. Installation

OptOP needs a Fortran compiler at build time — gfortran or Intel ifx/mpiifx. Everything else is handled by pip.

```
pip install git+https://github.com/EnCRiPT-IISc/Polymorph-Classfier.git
```

For MPI support, install the optional extra:

```
pip install "OptOP[mpi] @ git+https://github.com/EnCRiPT-IISc/Polymorph-Classfier.git"
```

Or from a local clone, editable:

```
git clone https://github.com/EnCRiPT-IISc/Polymorph-Classfier.git
cd Polymorph-Classfier
pip install -e .
pip install -e ".[mpi]"      # optional MPI support
```

There is no separate make step. The Fortran extension is compiled during installation by meson-python + f2py. If the build fails, check that a Fortran compiler is on your PATH.

Use Python 3.11 or 3.12. On Python 3.14 there are no prebuilt wheels yet for scipy, xgboost or mlxtend, so pip compiles them from source and installation can take well over ten minutes.

2. What OptOP Does

- **Computes 383 order parameters** per central atom or per molecule centre of mass.
- **Works on any LAMMPS trajectory** — water, ice, gas hydrates, minerals, metals.
- **Selects atoms by LAMMPS type number**, not element name.
- **Flexible neighbour definition** — same-type (O–O), mixed types with the molecular convention, or centre-of-mass to centre-of-mass.
- **Handles triclinic cells exactly**, with explicit control over the treatment.
- **Handles heterogeneous molecules** — different atom counts per molecule, e.g. water mixed with methane.
- **Two MPI axes** — over frames, and over atoms within a frame; both are bit-identical to a serial run.
- **Trains XGBoost classifiers** with importance ranking, randomised hyper-parameter search, and Sequential Forward Selection.

- Writes confusion matrices, accuracy curves and per-class reports for every model.

3. Commands

OptOP has three subcommands. Run `OptOP --help` for the full list of options.

Command	What It Does
<code>OptOP compute</code>	Compute order parameters from <code>.lammprj</code> trajectories
<code>OptOP train</code>	Train an XGBoost classifier on an existing OP CSV
<code>OptOP run</code>	Do both: compute order parameters, then train

The commands `optop` and `OP_ML` are aliases for `OptOP`; the last is kept so scripts written for the previous package keep working.

*Only the **compute** stage is MPI-parallel. Training always runs on a single rank. With `OptOP run` under MPI, computation is distributed and training then proceeds on rank 0.*

4. Compute Options

Option	Default	What It Does
<code>--traj</code>	(required)	Trajectory file, or directory of <code>*.lammprj</code> . Each filename becomes one class label.
<code>--central-type</code>	1	LAMMPS atom type of the central atom (e.g. 1 = O)
<code>--neighbor-types</code>	central only	Comma list of neighbour atom types, e.g. 1,2,3
<code>--com</code>	off	Reduce each molecule to its centre of mass (COM–COM mode)
<code>--masses</code>	auto	COM masses as <code>type:mass,...</code> else auto-detected from element symbols
<code>--rcut</code>	4.5	Neighbour cutoff in Angstrom
<code>--structure</code>	auto	Periodic-boundary treatment: auto, orthorhombic or triclinic (section 10)
<code>--atom-ranks</code>	1	MPI ranks per frame-group for 2-D decomposition (section 11)
<code>--op-categories</code>	all	Which families to compute: B,D,F,I,Q,W,LQ,LW or all
<code>--op-type</code>	avg	Which values to write: avg, local or both
<code>--last-frames</code>	all	Use only the last N frames of each trajectory
<code>--out-prefix</code>	OP	Output CSV prefix
<code>--outdir</code>	results	Output directory

5. Train Options

Option	Default	What It Does
<code>--data</code>	(required)	OP CSV containing a Class column
<code>--outdir</code>	ml_results	Where to write models, plots and reports
<code>--feature-mode</code>	cumulative	Top-feature selection: cumulative importance, or a fixed count
<code>--imp-threshold</code>	0.90	Cumulative-importance cutoff (cumulative mode)
<code>--max-top</code>	20	Hard cap on the number of top features kept
<code>--max-sfs</code>	6	Maximum k for Sequential Forward Selection

Option	Default	What It Does
--n-iter	25	RandomizedSearchCV iterations
--cv	5	Cross-validation folds
--overfit-gap	0.05	Train-test accuracy gap above which a model is flagged overfit
--param-grid	built-in	JSON file with your own XGBoost hyper-parameter grid
--phases	all	Comma list of Class labels to train on
--tag	outdir name	Label used in plot titles

6. Selecting Central and Neighbour Atoms

Atoms are chosen by their **LAMMPS type number** — the integer in the type column of the dump — not by element name. --central-type picks the atom whose environment is described; --neighbor-types picks which atoms may enter that environment. If --neighbor-types is omitted, only the central type is used.

System	Central	Neighbours	Flags
Water, O–O	1 (O)	1 (O only)	--central-type 1
Water, O sees O+H	1 (O)	1,2	--central-type 1 --neighbor-types 1, 2
Hydrate, O sees O+H+CH ₄	1 (O)	1,2,3	--central-type 1 --neighbor-types 1, 2, 3
CaCO ₃ , Ca–Ca	1 (Ca)	1	--central-type 1
CaCO ₃ , Ca sees Ca+C	1 (Ca)	1,3	--central-type 1 --neighbor-types 1, 3

The molecular convention. When neighbour types span more than the central type, guest species enter a central atom's *local* environment — they contribute to its neighbour list and therefore to its local Steinhardt sums — but they are excluded from the *averaging* set. So a water oxygen in a methane hydrate sees the methane when its own local order parameters are formed, yet Lechner–Dellago averaging runs only over water oxygens. This is what the reference implementation does.

7. Centre-of-Mass Mode

With --com each molecule is collapsed to a single centre-of-mass site and order parameters are computed COM–COM. Masses are auto-detected from element symbols in the dump when possible; otherwise give them explicitly:

```
# masses auto-detected
OptOP compute --traj phases/ --central-type 1 --com --outdir out/

# explicit per-type masses: O, H, CH4
OptOP compute --traj phases/ --central-type 1 --com \
  --masses 1:15.999,2:1.008,3:16.043 --outdir out/
```

*COM mode handles **heterogeneous molecules** correctly — molecules may contain different numbers of atoms, so water and methane can be mixed in one trajectory.*

8. Order-Parameter Families

All 383 order parameters are computed by default. Restrict with --op-categories, e.g. --op-categories Q,W,LQ,LW.

Category	Count	What It Is
Q	10	Steinhardt Q _l for l = 2,3,4,5,6,8,10,12,14,16
W	10	Steinhardt W _l (third-order invariant) for the same l
LQ	10	Locally averaged Q _l (Lechner–Dellago)
LW	10	Locally averaged W _l
B	42	Bond-angle order parameters
D	25	Distance-triplet parameters
F	275	Mixed distance–angle (Fourier) parameters
I	1	Jiang tetrahedral invariant
all	383	Everything (default)

```
# Steinhardt families only
OptOP compute --traj phases/ --central-type 1 --op-categories Q,W,LQ,LW --outdir out/

# bond-angle only
OptOP compute --traj phases/ --central-type 1 --op-categories B --outdir out/
```

Averaged versus local values

--op-type selects which values are written. avg (the default) writes the Lechner–Dellago averaged values, local writes the unaveraged per-molecule values, and both writes two CSVs. Averaged columns carry an avg suffix, e.g. Q₆avg.

9. Using Only Part of a Trajectory

If a trajectory has 2000 frames but only the equilibrated tail is wanted:

```
OptOP compute --traj phases/ --central-type 1 --last-frames 50 --outdir out/
```

10. Periodic Boundaries: Orthorhombic and Triclinic

Orthogonal cells are treated identically by all three modes, so --structure only matters for a tilted cell — one whose dump header reads ITEM: BOX BOUNDS xy xz yz ..., as in structure sH.

Mode	Behaviour
auto	Default. Applies the exact tilt-aware minimum image if and only if the dump header declares tilt factors; otherwise treats the cell as orthogonal.
orthorhombic	Ignores tilt entirely and uses the raw (lo, hi) bounds as edge lengths. This reproduces the original reference code, which read only the two lo/hi numbers and used a diagonal lattice even for sH.
triclinic	Always converts the LAMMPS bounding box back to the true cell edges and keeps xy, xz, yz, giving the physically exact minimum image. Identical to orthorhombic for an untilted cell.

*For a triclinic dump LAMMPS writes the **bounding** box, not the cell edges. auto and triclinic invert that transformation before applying the minimum image. Use orthorhombic only when you need to reproduce older reference numbers.*

11. Running in Parallel

11.1 Frame decomposition (default)

By default MPI distributes whole frames round-robin across ranks. This is the right choice when there are many frames.

```
mpiexec -n 8 OptOP compute --traj phases/ --central-type 1 --rcut 4.5 --outdir out/
mpiexec -n 48 OptOP compute --traj phases/ --central-type 1 --rcut 4.5 --outdir out/
```

11.2 Two-dimensional decomposition: frames × atoms

With `--atom-ranks P` the `N` ranks are split into $G = N/P$ frame-groups of `P` ranks each. Frames are distributed round-robin across the `G` groups, and within a group the `P` ranks divide up that frame's central molecules, cooperating through two MPI all-reduces per frame. This adds a second parallel axis, which helps when there are few frames but each is very large.

```
# 48 ranks = 12 frame-groups x 4 atom-ranks
mpiexec -n 48 OptOP compute --traj phases/ --central-type 1 \
  --neighbor-types 1,2,3 --rcut 4.5 --atom-ranks 4 --outdir out/
```

Ranks	--atom-ranks	Layout	Best when
8	1	8 frame-groups x 1	Many frames (the usual case)
48	4	12 frame-groups x 4	Few frames, many molecules each
48	48	1 frame-group x 48	A single very large frame

*The total number of ranks must be an exact multiple of `--atom-ranks`, otherwise every rank raises an error. Results are **bit-identical** to frame-only and serial runs, because each molecule is computed independently and the out-of-slice contributions are exactly zero.*

12. Output Files

12.1 From OptOP compute

File	What It Contains
OP.csv	Averaged order parameters for every phase, with <code>mol_id</code> and <code>Class</code> columns
OP_local.csv	Unaveraged (local) order parameters — written when <code>--op-type</code> is local or both

The prefix follows `--out-prefix`, so `--out-prefix Hydrate` gives `Hydrate.csv`.

12.2 From OptOP train

File	What It Contains
model_summary.csv	One row per SFS model $k = 1..max_sfs$: features, accuracies, CV score, overfit flag
stages.csv	Baseline and top-N stage accuracies with train-test gaps
best_model.json	Chosen k , its features, test and CV accuracy, and best hyper-parameters
best_params.json	Winning XGBoost hyper-parameters from RandomizedSearchCV
best_classification_report.csv	Per-class precision, recall and F1 for the chosen model
feature_importances.csv / .png	Every feature ranked by gain, with cumulative importance and a selected flag
class_mapping.json	Class label to integer encoding
cm_baseline, cm_topN, cm_kfeat	Confusion matrices as both .csv and .png, for every stage and every k
accuracy_curve.png	Train, test and CV accuracy against k

File	What It Contains
sfs_stddev.png	Sequential Forward Selection score with confidence bands
hist_1feat.png	Per-class distribution of the single best order parameter
scatter_2feat.png / scatter_3feat.png	Class separation in the best two and three order parameters
pca_kfeat.png	Two-component PCA projection for $k \geq 4$
xgb_baseline.pkl, xgb_topN.pkl, xgb_kfeat.pkl	Pickled trained models, one per stage and per k

13. Worked Example: Hydrate and Ice Classification

Step 1 — lay out the trajectories

One file per phase; the filename becomes the class label.

```
phases/
  Ice1c.lampstrj
  Ice1h.lampstrj
  Liquid.lampstrj
  S1.lampstrj
  S2.lampstrj
  SH.lampstrj
```

Step 2 — compute order parameters

```
# water 0 as central, seeing 0, H and methane; last 50 frames; 48 ranks
mpiexec -n 48 OptOP compute --traj phases/ --central-type 1 \
  --neighbor-types 1,2,3 --rcut 3.5 --op-categories Q,W,LQ,LW \
  --last-frames 50 --outdir out/
```

Step 3 — train a classifier

```
OptOP train --data out/OP.csv --outdir out/ml/ \
  --feature-mode cumulative --imp-threshold 0.90 --max-top 20 \
  --phases Ice1c,Ice1h,S1,S2,SH --n-iter 25 --cv 5 --max-sfs 6
```

Step 4 — read the results

```
cat out/ml/best_model.json      # chosen k, its features, accuracy
cat out/ml/model_summary.csv   # every k, with the overfit flag
```

Prefer the highest test accuracy whose overfit flag is false. The gap threshold is set by `--overfit-gap`.

Step 5 — use a trained model

```
import pickle
model = pickle.load(open('out/ml/xgb_3feat.pkl', 'rb'))
predictions = model.predict(X_new)
```

Or do it all in one command

```
mpiexec -n 48 OptOP run --traj phases/ --central-type 1 \
  --neighbor-types 1,2,3 --rcut 3.5 --op-categories Q,W,LQ,LW \
  --last-frames 50 --max-sfs 6 --outdir out/
```

run writes order parameters to out/OP.csv and training results to out/ml/.

14. Custom Hyper-parameter Grid

Pass a JSON file to override the built-in XGBoost search grid:

```
# my_params.json
{
  "n_estimators": [100, 500, 1000],
  "learning_rate": [0.01, 0.05, 0.1],
  "max_depth": [3, 5, 10],
  "subsample": [0.5, 0.7, 0.9]
}
```

```
OptOP train --data out/OP.csv --param-grid my_params.json --n-iter 50 --outdir ml/
```

15. Migrating from OP_ML

This repository previously published the OP_ML distribution. The table below lists what changed.

	OP_ML (previous)	OptOP (current)
Distribution	OP_ML	optop
Import	import op_ml	import optop
Command	OP_ML	OptOP (OP_ML kept as an alias)
Build backend	hatchling	meson-python
Fortran build	manual make, or lazy auto-build	compiled during pip install
MPI axes	frames only	frames, and frames x atoms
Periodic boundaries	implicit	explicit --structure flag
Subcommands	compute, train, run, validate	compute, train, run

Behaviour change worth knowing

The fourth bond-angle order parameter is now $\text{PI}/4$, not $\text{PI}/3$, which matches the original reference implementation. Bond-angle values computed with the older package differ in that one column, so **do not mix** feature tables produced by the two versions.

Features not carried over

- **validate** — the standalone NaN-checking subcommand. OptOP instead fails during computation rather than offering a separate pass.
- **Interactive prompts** — phase merging and parameter confirmation. Every setting is now given on the command line, so runs are reproducible and script-friendly. To merge phases, edit the Class column of the OP CSV, or select a subset with --phases.

If either of these is in active use, raise it with the maintainers before switching — they can be ported.

16. Troubleshooting

Problem	What To Do
Fortran extension not found at run time	Reinstall; the extension is built during pip install. Check that gfortran or ifx is on your PATH, and read the pip output for the meson build error.

Problem	What To Do
Build fails with UnicodeDecodeError	Fixed in the current version. If you see it in an older checkout, the cause is a non-ASCII byte in a Fortran comment read under a C locale; upgrade.
--atom-ranks error about rank count	The total number of MPI ranks must be an exact multiple of --atom-ranks.
NaN values in the output	Almost always isolated atoms with no neighbours inside --rcut. Increase the cutoff, or check the frame for stray molecules.
Order parameters differ from older reference data	Try --structure orthorhombic, and note the $\pi/4$ bond-angle change in section 15.
Wrong atoms selected	--central-type takes a LAMMPS type number, not an element symbol. Check the type column of your dump.
Segfault under MPI	Retry with fewer ranks, and with --atom-ranks 1, to isolate which axis is at fault.
Very slow install	Python 3.14 has no prebuilt scipy/xgboost/mlxtend wheels. Use Python 3.11 or 3.12.

Need more detail? Run `OptOP --help`, `OptOP compute --help` or `OptOP train --help`. Source and issue tracker: <https://github.com/EnCRIPT-IIsc/Polymorph-Classifer>